

Flex — 제약을 숨기지 않는 근무 계획기

2026.07 — 진행 중 · 개인 설계·구현·검증 · 개인

[GitHub](#) [상세](#)

결정론적 월간 allocator · 날짜별 제약 · Python/React 이중 검증

인정시간·실제 근무·휴가·일/주 상한을 하나의 결정론적 배분 계약으로 묶고, 달성 불가능한 목표는 초과 배정 대신 부족분과 이유로 표시하는 로컬 우선 근무 계획기

Python React TypeScript FastAPI Playwright Property Testing GitHub Actions

Flex constraint planning boundary

Synthetic-only evidence · no external HR-system writes



Candidate evidence: default-branch SHA + Python/browser CI + deterministic fixtures

불가능한 목표를 숨기지 않는 플래너

Preserve hard limits; expose the remaining gap and reason.



Synthetic scenario · 실제 근무기록 아님 / not a real work record

합성 월 목표가 남은 일·주 상한보다 큰 경우 상한을 유지하고 insufficient_slots와 부족 시간을 반환합니다.

성과 지표	이전	이후
불가능 상태	강제 배정 위험	insufficient_slots + gap (hard limit 유지)
날짜 변경 범위	월 전체 재계산 위험	고정 날짜 + 나머지 재배포 (preview·재배포 경계)

문제 해결 과정

목표 시간이 남은 가용 슬롯보다 커도 강제로 초과 배정하면 계획이 현실을 숨기는 문제

- 일·주 상한과 고정 계획을 먼저 적용하고 배정 가능한 시간을 계산한 뒤 부족분을 명시적 상태로 반환
- 상한을 깨지 않고 **insufficient_slots**와 부족 시간을 화면에 표시

특정 날짜 변경이 월 전체 계획을 예기치 않게 덮어쓸 수 있는 문제

- 고정 날짜를 우선 잠그고 충돌 preview 후 나머지 날짜에만 결정론적 재배포 적용
- 외부 시스템 쓰기 없이 **preview**에서 변경 범위와 충돌 경계를 표시

기술 선택 근거

- 기존 `_minimum_activation` 흐름을 복원해 최소 근무시간을 의무 시드가 아닌 선택적 활성화 하한으로 유지
- 외부 시스템 연동을 추가하지 않고 URL/로컬 상태로 재현 가능한 판단 표면 제공

주요 내용

- 고정 계획 우선·선택적 최소 활성화·hard-limit 유지의 단일 allocator 계약
- 달성 불가능 상태를 부족 시간과 원인으로 노출
- 날짜 override preview와 나머지 날짜 재배포 계산
- synthetic scenario·공개 경계 scan·Python/Playwright CI

깨달은 점

- 최적화보다 먼저 불가능 상태를 정확히 모델링해야 계획기가 현실을 왜곡하지 않는다.
- 사용자 override는 전체 재계산보다 고정 범위와 나머지 배분 경계를 분리해야 예측 가능하다.

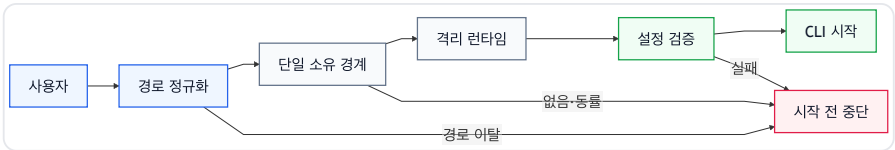
프로젝트별 AI CLI 런타임을 분리하고 검증하는 공개 런처

2026.04 — 진행 중 · 개인 설계·구현·운영 · 개인

[GitHub](#) ↗ [Demo](#) ↗ [v0.20.0 Release](#) ↗ [CI](#) ↗ [상세](#) ↗

공개 런처 · AI CLI 3종 · contract demo

Claude Code·Codex·Kiro의 전역 상태를 프로젝트별 실행 경계로 분리하고, 잘못된 작업 경계와 설정 드리프트를 시작 전에 차단하는 공개 런처입니다.



성과 지표	이전	이후
런타임 격리	프로젝트가 전역 CLI 상태를 공유	등록 프로젝트별 런타임 홈 (세션·스킬·MCP·정책을 작업 경계별로 분리)
경계 선택	이름·원격 저장소 기반 프로파일 추정 위험	정규화 경로의 단일 소유 경계 선택 (경계 밖·이탈·모호함은 AI CLI 시작 전 중단)
공개 검증	비공개 환경 설명에 의존	공개 launcher + contract demo + CI (로그인 없이 핵심 계약과 실패 동작 재현)

문제 해결 과정

AI CLI의 전역 홈을 여러 프로젝트가 공유하면 세션·도구·규칙이 다른 작업 경계로 섞일 수 있었습니다.

- 등록한 프로젝트마다 Codex·Kiro 런타임 홈을 생성하고, 프로파일 명령과 harness-exec가 같은 실행 정책을 사용하도록 통합했습니다.
- **프로젝트별 런타임 상태를 분리하고, 공개 저장소의 설치·프로파일·런타임 테스트로 동작을 검증합니다.**

워크트리 실행기가 프로필을 이름이나 원격 저장소로 추정하면 잘못된 권한·도구 경계에서 에이전트를 시작할 수 있었습니다.

- harness-auto가 현재 경로를 정규화하고 가장 구체적인 단일 등록 경계만 선택하도록 했습니다. 경계 밖·심볼릭 링크 이탈·동일 우선순위 중복은 거부합니다.
- **프로필 추정 대신 경로 소유권을 사용하며, 선택이 확정되지 않으면 AI CLI를 실행하지 않습니다.**

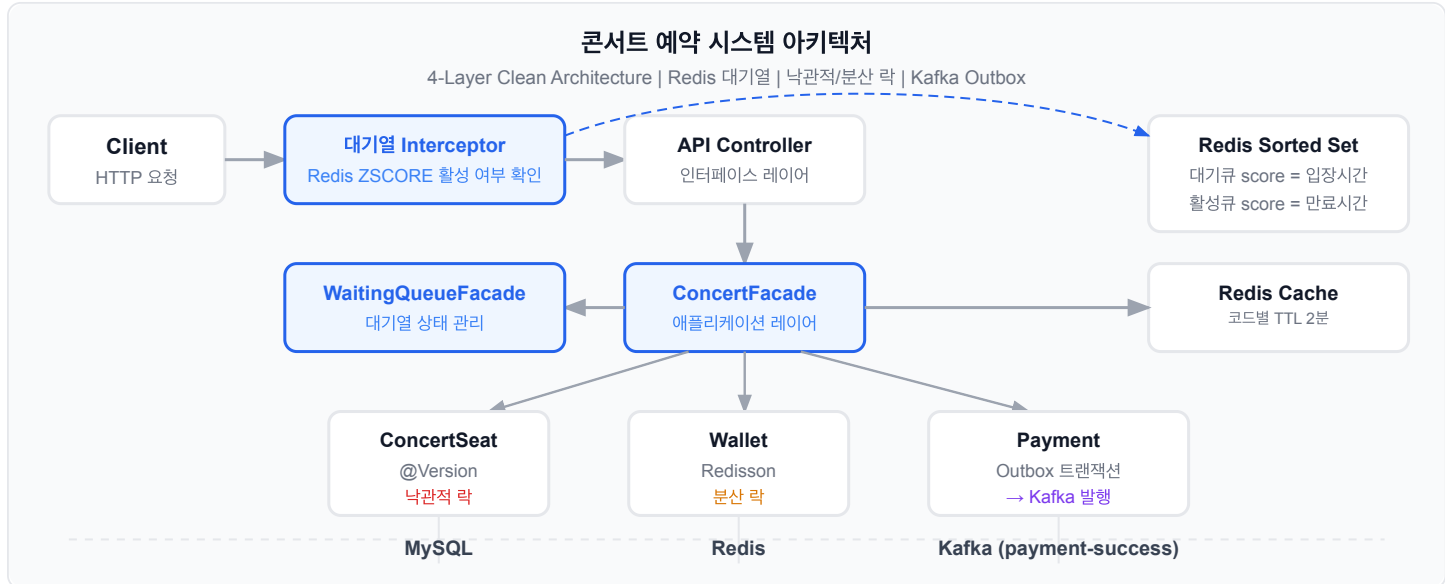
비공개 운영 환경만으로 설명하면 방문자가 프로파일 분리와 근거 재확인 계약을 독립적으로 검증할 수 없었습니다.

- 공개 harness-launcher 구현·테스트와 별도 public contract demo를 연결했습니다. 데모는 프로파일 고유성, 경로 이탈, 원문 SHA-256 드리프트, 관계 무결성을 표준 라이브러리 테스트로 확인합니다.
- **로그인 없이 README·소스·CI에서 핵심 경계와 실패 동작을 재현할 수 있으며, 비공개 회사 데이터와 운영 인덱스는 공개 근거에서 제외했습니다.**

k6 설정값 3,000 requests/s · 최대 2,400 VU · 관측 예약 오류율 0.64% · Prometheus+Grafana

동일 좌석·사용자 지갑 동시성 테스트로 좌석에는 낙관적 락, 결제·충전에는 사용자 단위 Redisson 락을 적용하고, 공개 k6 보고서에서 캐시 스탬피드 p95를 5.74초에서 676ms로 낮춘 콘서트 예약 프로젝트

Java Spring Boot JPA Redis MySQL Kafka Docker K6



성과 지표	이전	이후
예약 p95	5.74s	676ms (캐시 워밍 + TTL 2분 → 5분)
예약 오류율	-	0.64% (예약 시나리오 394건 실패 기록)
대기열 p95	-	62.5ms (대기열 시나리오 오류율 0%)

문제 해결 과정

동일 좌석에 100개 thread가 동시에 예약을 시도할 때 중복 예약 위험

ConcertSeat @Version 낙관적 락과 100-thread Spring 통합 테스트로 충돌 경로 검증

→ 테스트 종료 후 예약 행 1개와 reserved 상태를 assertion으로 확인

포인트 충전과 결제의 동시 실행 시 잔액 정합성 붕괴

Redisson 분산 락(userWalletLock:{userId})으로 사용자 단위 충전·결제 경로를 직렬화

→ 100-thread 충전 테스트에서 최종 잔액을 기대 합계와 비교하고 결제 테스트에서 단일 결제 상태를 확인

예약 부하 테스트에서 캐시 만료 구간의 stampede와 p95 5.74s 지연 관측

예약 시작 3분 전 캐시 워밍과 TTL 2분 → 5분 조정을 적용

→ 공개 보고서 기준 예약 p95 5.74s → 676ms, 설정 SLO 1s 이내

결제 이벤트의 저장·발행·실패 재시도 상태를 하나의 동기 호출에서 분리해야 했다

Kafka Transactional Outbox 패턴으로 결제 TX 내 PENDING outbox_event를 저장하고 scheduler가 비동기 발행

→ 결제 트랜잭션의 PENDING outbox 생성과 발행 실패 상태·재시도 경로를 코드와 테스트로 분리

기술 선택 근거

- 좌석 선점에 @Version 낙관적 락: 100-thread 동일 좌석 통합 테스트가 예약 행 1개를 assertion
- 결제·충전에 Redisson 사용자 단위 락: 100-thread 테스트가 결제 상태와 최종 잔액을 기대값과 비교
- Redis Sorted Set 대기열: 입장 순서와 만료 상태를 score로 관리
- 결제 TX에서 PENDING outbox 생성 후 scheduler가 발행 실패 상태와 재시도 경로를 관리

주요 내용

- 동일 좌석 100-thread 통합 테스트에서 예약 행 1개 확인
- 사용자 지갑·결제를 Redisson 사용자 단위 락으로 직렬화
- 캐시 워밍·TTL 조정 후 공개 보고서 p95 5.74s → 676ms
- 결제 TX의 PENDING outbox와 scheduler 발행·재시도 경로 구현
- k6 설정값 3,000 requests/s와 최대 2,400 VU를 관측 성과와 분리

깨달은 점

- 동일 좌석과 사용자 지갑의 경험 범위에 따라 낙관적 락과 사용자 단위 분산 락을 구분
- k6 설정값·최대 VU와 보고서의 관측 결과를 분리해 성능 수치의 범위를 명확히 기록

[라이브 데모 ↗](#) [GitHub ↗](#) [상세 ↗](#)

공개 가능한 UI 26개 파일을 hash manifest로 고정하고, 합성 seed와 브라우저 localStorage 어댑터를 연결해 실제 가족 데이터·운영 DB·자격증명 없이 7개 공개 화면의 입력·조화·초기화를 재현했다.

TypeScript Next.js React Vitest Vercel

